FCEFyN
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Redes de Computadoras

Trabajo Práctico

N.º 7(Práctico)

Desafío final: captura, de-codificación y recuperación de datos en redes de computadoras

Grupo: NetRunners
Docente: SANTIAGO MARTIN HENN

Integrantes

Bastida, Lucas Ramiro	DNI 39444511
Contessi, Ruben Andres	DNI 33315235
Garay, Ignacio Hernan	DNI 44191925
Giraudo, Juan Pablo	DNI 34970089
Peñaloza, Gonzalo Adrian	DNI 40441355
Quinteros del Castillo, Tomás Agustín	DNI 46169739
Vasconsellos Blason, Benjamin Alejandro	DNI 45642879

3 de noviembre de 2026

Ejercicio 1

Averigüen su IP y su máscara (`ipconfig` / `ip addr`) y calculen la dirección de broadcast de la red. ¿Cuál es la dirección MAC de broadcast?

Conectados a la red Wi-Fi de la cátedra por DHCP, ejecutamos `ipconfig` en Windows. El dato relevante es el del adaptador de LAN inalámbrica Wi-Fi:

Dato	Valor
Dirección IPv4	192.168.8.182
Máscara de subred	255.255.255.0 (/24)
Puerta de enlace	192.168.8.1

El adaptador "Ethernet 3" (192.168.56.1) es una interfaz virtual de la PC y no participa de la red del aula.

Cálculo de la dirección de broadcast. La máscara /24 indica que los primeros 24 bits son de red y los últimos 8 son de host:

	Decimal	Binario
IP	192.168.8.182	`11000000.10101000.00001000.10110110`
Máscara	255.255.255.0	`11111111.11111111.11111111.00000000`
Red (IP AND máscara)	192.168.8.0	`11000000.10101000.00001000.00000000`
Broadcast (bits de host en 1)	192.168.8.255	`11000000.10101000.00001000.11111111`

- **Dirección de red:** 192.168.8.0
- **Dirección de broadcast:** 192.168.8.255. Coincide con la IP destino de los paquetes del transmisor, lo que confirma la máscara.
- **Rango de hosts utilizables:** 192.168.8.1 a 192.168.8.254
- **Dirección MAC de broadcast:** `ff:ff:ff:ff:ff:ff`, los 48 bits en 1. Todas las placas de la red local aceptan las tramas dirigidas a esa dirección.

Ejercicio 2

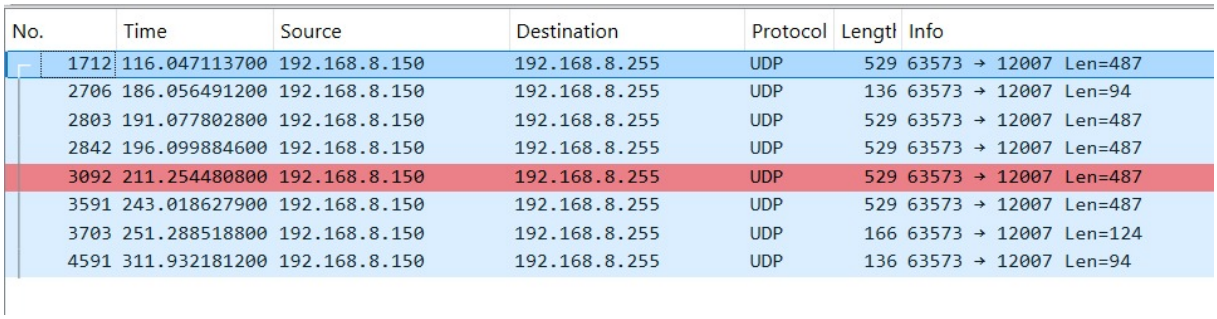
Abran Wireshark en la interfaz Wi-Fi y capturen al menos 2 minutos. Pueden llegar varios mensajes; no tienen por qué ser todos idénticos.

Capturamos en la interfaz Wi-Fi durante más de dos minutos. Para aislar los mensajes del transmisor usamos el filtro `frame contains "SIZE"`, ya que la consigna adelantaba que la carga era un JSON con ese campo. También sirve `eth.dst == ff:ff:ff:ff:ff:ff && udp`, que muestra todo el tráfico de difusión. La captura se guardó en formato `.pcapng` y se entrega junto con el informe.

En la primera captura (de los 116 s a los 312 s) llegaron 8 datagramas del transmisor, todos de 192.168.8.150:63573 a 192.168.8.255:12007, con tres tamaños de carga distintos (Figura 1):

Carga	Tramas	Contenido
487 bytes	1712, 2803, 2842, 3092, 3591	Mensaje 1 (SIZE 8,7), repetido aprox. cada 30 s
94 bytes	2706, 4591	[Completar]
124 bytes	3703	[Completar]

Detalle del primer mensaje (trama 1712):



No.	Time	Source	Destination	Protocol	Lengtl	Info
1712	116.047113700	192.168.8.150	192.168.8.255	UDP	529	63573 → 12007 Len=487
2706	186.056491200	192.168.8.150	192.168.8.255	UDP	136	63573 → 12007 Len=94
2803	191.077802800	192.168.8.150	192.168.8.255	UDP	529	63573 → 12007 Len=487
2842	196.099884600	192.168.8.150	192.168.8.255	UDP	529	63573 → 12007 Len=487
3092	211.254480800	192.168.8.150	192.168.8.255	UDP	529	63573 → 12007 Len=487
3591	243.018627900	192.168.8.150	192.168.8.255	UDP	529	63573 → 12007 Len=487
3703	251.288518800	192.168.8.150	192.168.8.255	UDP	166	63573 → 12007 Len=124
4591	311.932181200	192.168.8.150	192.168.8.255	UDP	136	63573 → 12007 Len=94

Figure 1: Datagramas UDP del transmisor capturados en Wireshark.

Capa	Campo	Valor
Trama	Tamaño	529 bytes (4232 bits)
Ethernet II	MAC origen	`dc:90:09:ed:91:35` (Intel)
Ethernet II	MAC destino	`ff:ff:ff:ff:ff:ff` (broadcast)
IPv4	IP origen	192.168.8.150
IPv4	IP destino	192.168.8.255 (broadcast de la red)
UDP	Puerto origen / destino	63573 / 12007
Datos	Carga útil	487 bytes de texto ASCII

Los tamaños cierran: 14 bytes de Ethernet + 20 de IPv4 + 8 de UDP + 487 de datos = 529 bytes. Que el transporte sea UDP tiene sentido, porque TCP no puede hacer broadcast: una conexión TCP siempre es entre dos extremos.

Para extraer la carga útil hicimos clic derecho sobre la capa *Data* → *Copiar* → *. . . as ASCII Text*.

Más tarde el transmisor envió una segunda tanda con otros dos mensajes válidos (mensajes 2 y 3), que también capturamos. Consideramos válido un paquete que trae los campos SIZE, PL y CHECK y cuyos largos cierran con SIZE (ver punto 3); los demás los descartamos. En total analizamos **tres mensajes válidos distintos**:

Mensaje	SIZE	Largo de PL	Largo de CHECK	Datos del JSON
1	8,7	392	57	487 bytes
2	10,6	420	52	511 bytes
3	4,4	112	32	182 bytes

Ejercicio 3

Analicen la estructura del mensaje. ¿Qué formato tiene? ¿Qué campos contiene y qué creen que representa cada uno? Con lo visto en la sección 1, propongan cómo hay que interpretar los datos.

La carga útil es un objeto **JSON** con tres campos de texto:

```
{"SIZE": "8,7", "PL": "110100011101...", "CHECK": "011111010110..."}
```

Campo	Qué representa
SIZE	Tamaño de la matriz, en formato FILAS,COLUMNAS. Las columnas se cuentan en **caracteres**.
PL	La carga (*payload*): los bits de datos, fila por fila.
CHECK	Los bits de control: primero la paridad par de cada fila (de arriba hacia abajo) y después la de cada columna (de izquierda a derecha).

Interpretación propuesta, según la sección 1:

1. Cada carácter ocupa 7 bits ASCII, así que una fila de C caracteres tiene $7C$ bits. Con SIZE "8,7" quedan 8 filas de 7 caracteres, o sea 8 filas de 49 bits.
2. El largo de PL tiene que ser filas $\times$ 7 $\times$ columnas: $8 \times 49 = 392$ bits.
3. El largo de CHECK tiene que ser filas $+ 7 \times$ columnas: $8 + 49 = 57$ bits (8 de paridad de filas y 49 de columnas).
4. Se corta PL en filas, se recalcula la paridad par de cada fila y de cada columna, y se compara con CHECK. Las filas y columnas que no coinciden forman el síndrome.
5. Después de corregir, cada fila se decodifica de a 7 bits con `decodificar_mensaje.py`.

Los tres mensajes cumplen estas reglas: $392 = 8 \times 49$ y $57 = 8 + 49$; $420 = 10 \times 42$ y $52 = 10 + 42$; $112 = 4 \times 28$ y $32 = 4 + 28$.

Un detalle: los unos y ceros viajan como caracteres ASCII ("1" y "0", un byte cada uno), no como bits reales. Es solo por el valor didáctico del laboratorio; un código de detección y corrección de errores real opera directamente sobre los bits.

Ejercicio 4

Verifiquen la integridad del mensaje con el código de la sección 1: recalculen las paridades y compárenlas con las recibidas. Registren qué filas y qué columnas fallan (el síndrome).

Procedimiento:

1. Para cada mensaje cortamos PL en filas según SIZE (7 bits por carácter).
2. Calculamos la paridad par de cada fila y de cada columna con `paridad.py`.
3. Comparamos el resultado con CHECK. Sus primeros bits son las paridades de las filas, de arriba hacia abajo, y el resto las de las columnas, de izquierda a derecha.

Las filas y columnas donde no coinciden forman el **síndrome**. En las comparaciones de columnas, las paridades van agrupadas de a 7 (un carácter por grupo) y ^ marca cada columna que falla.

Mensaje 1 (SIZE 8,7: 8 filas de 49 bits; CHECK = 8 + 49 bits)

Fila	Calculada	Recibida	Resultado
1	0	0	OK
2	1	1	OK
3	1	1	OK
4	1	1	OK
5	0	1	**Falla**
6	0	1	**Falla**
7	1	0	**Falla**
8	1	1	OK

```
Columnas calculadas: 0110111 1011101 1011110 0001111 0011000 0110011 0010001
Columnas recibidas:  0110111 1111101 1001110 0101111 0011000 0110011 0010001
                     ....... .^..... ..^.... .^..... ....... ....... .......
```

Síndrome: filas 5, 6 y 7; columnas 9, 17 y 23.

Mensaje 2 (SIZE 10,6: 10 filas de 42 bits; CHECK = 10 + 42 bits)

Fila	Calculada	Recibida	Resultado
1	0	1	**Falla**
2	0	0	OK
3	0	0	OK
4	0	0	OK
5	1	0	**Falla**
6	1	0	**Falla**
7	0	0	OK
8	0	1	**Falla**
9	0	0	OK
10	0	0	OK

```
Columnas calculadas: 1000011 1110100 1010110 1110001 1010100 1011001
Columnas recibidas:  1010011 1110000 1011110 1111001 1010100 1011001
                     ..^.... ....^.. ...^... ...^... ....... .......
```

Síndrome: filas 1, 5, 6 y 8; columnas 3, 12, 18 y 25.

Mensaje 3 (SIZE 4,4: 4 filas de 28 bits; CHECK = 4 + 28 bits)

Fila	Calculada	Recibida	Resultado
1	0	1	**Falla**
2	1	0	**Falla**
3	1	0	**Falla**
4	1	0	**Falla**

```
Columnas calculadas: 1110101 0110010 0000101 0100000
Columnas recibidas:  1110100 0111010 1000111 0100000
                     ......^ ...^... ^....^. .......
```

Síndrome: las 4 filas; columnas 7, 11, 15 y 20.

Conclusión. Ningún mensaje llegó intacto. En los tres, la cantidad de filas que fallan es igual a la de columnas que fallan (3 y 3, 4 y 4, 4 y 4), y no hay filas sin columna ni columnas sin fila. Eso indica errores en bits de datos, uno por fila y cada uno en una columna distinta, y no en los bits de control: un bit de control dañado haría fallar una sola fila o una sola columna, sin su pareja. Como hay más de un error por mensaje, el síndrome dice qué filas y columnas están afectadas, pero no cómo se emparejan. Eso se resuelve en el punto 5.

Ejercicio 5

Si hay errores, ubíquenlos y corríjanlos. Para cada bit corregido indiquen fila, columna, valor recibido y valor corregido. Si recibieron más de un mensaje distinto, analicen cada uno por separado.

Los tres mensajes tenían **más de un error**, uno por fila y cada uno en una columna distinta. Con un solo error, el bit dañado estaría en el cruce de la única fila y la única columna que fallan. Con varios errores, la paridad dice qué filas y qué columnas fallan, pero no qué columna va con cada fila. Con k errores hay $k!$ formas de emparejarlas (6 con 3 errores, 24 con 4) y **todas dejan la paridad consistente**, porque cada una invierte exactamente un bit en cada fila y cada columna que falla. Para elegir, decodificamos cada candidato y nos quedamos con la única combinación que da un texto válido.

Para ubicar el carácter afectado: la columna c corresponde al carácter $\lceil c/7 \rceil$ de la fila, en su bit $(c-1) \bmod 7 + 1$.

Mensaje 1 (SIZE 8,7)

Síndrome: filas 5, 6 y 7; columnas 9, 17 y 23.

Sin corregir se leía `https://youtube.com/shorts/q8wgKzJwPa$?featurE=share`:

- En la fila 7, solo la columna 23 arregla la palabra ("featurE" → "feature").
- En la fila 6, la columna 23 da un carácter no imprimible y la columna 9 deja el `$`, que no es válido en una URL; solo la columna 17 lo convierte en "4".
- A la fila 5 le queda la columna 9.

Fila	Columna	Recibido	Corregido	Carácter
5	9	1	0	`1110111` (w) → `1010111` (W)
6	17	0	1	`0100100` ($) → `0110100` (4)
7	23	0	1	`1000101` (E) → `1100101` (e)

Mensaje 2 (SIZE 10,6)

Síndrome: filas 1, 5, 6 y 8; columnas 3, 12, 18 y 25.

Sin corregir: `xttps://youtube.com/shortw/WCi5h}1EqqI?featurm=share`:

- La fila 1 solo se arregla con la columna 3 ("xttps" → "https").
- La fila 8, con la columna 25 ("featurm" → "feature").
- La fila 5, con la columna 12 ("shortw" → "shorts").
- A la fila 6 le queda la columna 18, que cambia la `}` (inválida en una URL) por "u".

Fila	Columna	Recibido	Corregido	Carácter
1	3	1	0	`1111000` (x) → `1101000` (h)
5	12	1	0	`1110111` (w) → `1110011` (s)
6	18	1	0	`1111101` (}) → `1110101` (u)
8	25	1	0	`1101101` (m) → `1100101` (e)

Mensaje 3 (SIZE 4,4)

Síndrome: las 4 filas; columnas 7, 11, 15 y 20.

Sin corregir: `Qr3mi8!:)1p-dIgZ`. No es una URL ni una palabra, así que probamos las 24 combinaciones. Casi todas dan texto sin sentido (por ejemplo, `Qz3mh8!:)10-dIeZ` o `Pr3mi8a:)9p-dIeZ`). Solo una da un mensaje legible, escrito en *leet* (números en lugar de letras): `Pr3mi0!:)10-dIeZ`, es decir, "Premio! :) 10 diez".

Fila	Columna	Recibido	Corregido	Carácter
1	7	1	0	`1010001` (Q) → `1010000` (P)
2	11	1	0	`0111000` (8) → `0110000` (0)
3	15	1	0	`1110000` (p) → `0110000` (0)
4	20	1	0	`1100111` (g) → `1100101` (e)

En los tres casos, después de corregir volvimos a correr `paridad.py` y todas las filas y columnas coinciden con CHECK.

Ejercicio 6

Construyan un archivo de texto en el formato de mensaje.txt con los datos ya corregidos y de-codifíquenlo con decodificar_ mensaje.py (o el cuaderno). Si todo salió bien, el mensaje les va a indicar cómo seguir. Ojo: un mensaje que "se lee bien" no es necesariamente un mensaje correcto.

Con los bits corregidos armamos un archivo por mensaje en el formato de `mensaje.txt`, con una fila de la matriz por línea. Después los decodificamos con `python decodificar_mensaje.py corregidoN.txt`.

Mensaje 1 (`corregido1.txt`):

```
11010001110100111010011100001110011011101001011111
01011111111001110111111101011110100111010111000010
11001010101110110001111011111101101010111111110011
11010001101111111001011101001110011010111111110001
01110001010111110011110010111111010100101011101111
10100001100001011010001111111100110110010111000001
11101001110101111001011001010111101111001111101000
11000011110010110010101000000100000010000000100000
```

Salida: `https:/`, `/youtub`, `e.com/s`, `horts/q`, `8WgKzJw`, `Pa4?fea`, `ture=sh`, `are` (más espacios de relleno). Mensaje: `https://youtube.com/shorts/q8WgKzJwPa4?feature=share`

Mensaje 2 (`corregido2.txt`):

```
11010001110100111010011100001110011011101O
01011110101111111100111011111110101111O100
11101011100010110010101011101100011110111l
11011010101111111001111010001101111111OO10
11101001110011010111110101111000011110100l
01101011101000111010101100011000101111OOO1
11100011001001011111111001101100101110000l
11101001110101111001011001010111101111OO11
11010001100001111001011001010100000010000O
01000000100000010000001000000100000010000O
```

Mensaje: `https://youtube.com/shorts/WCi5hu1EqqI?feature=share`. La última fila es solo de espacios de relleno.

Mensaje 3 (`corregido3.txt`):

```
10100001110010011001111011O1
11010010110000010000101110l0
01010010110001011000000101101
11001001001001110010110110l0
```

Mensaje: `Pr3mi0!:)10-dIeZ` ("Premio! :) 10 diez").

Reflexiones e investigación

Reflexión 1

¿Qué hubiera pasado si el mensaje tenía errores en dos bits de filas y columnas distintas? ¿Y si se dañaba un bit de los de control?

Dos errores en filas y columnas distintas. Supongamos errores en (fila 2, columna 3) y (fila 5, columna 8). Fallan 2 filas (2 y 5) y 2 columnas (3 y 8), lo que deja 4 cruces candidatos: (2,3), (2,8), (5,3) y (5,8). Los dos errores pueden ser $\{(2,3),(5,8)\}$ o $\{(2,8),(5,3)\}$, y cualquiera de los dos pares, al invertirse, deja todas las paridades consistentes. El código **detecta** que hubo errores pero **no puede corregirlos** con certeza: hace falta otra información, como el contexto o una retransmisión. Es lo que nos pasó, ampliado, con los 3 y 4 errores de nuestros mensajes (6 y 24 combinaciones posibles).

Hay dos variantes peores:

- **Dos errores en la misma fila:** esa fila no aparece en el síndrome (sus dos cambios se cancelan) y solo fallan dos columnas, así que no se sabe en qué fila están.
- **Cuatro errores en las esquinas de un rectángulo:** se cancelan en todas las filas y columnas y no se detectan.

Un bit de control dañado. Falla solo una fila (si era una paridad de fila) o solo una columna (si era de columna), sin su pareja. Esa asimetría lo delata: los datos están bien y no hay que corregir nada. En nuestros tres mensajes no pasó, porque siempre fallaron tantas filas como columnas.

Reflexión 2

Si el mensaje es texto en castellano, muchas veces un error "se adivina" por contexto (Hola mondo). ¿Sirve eso para una contraseña, un número de cuenta o una dirección web? ¿Qué aporta el código frente a "leer y adivinar"?

No sirve. En un texto en castellano la redundancia del idioma permite adivinar, pero en una contraseña o un número de cuenta cualquier combinación es igual de válida: un dígito cambiado sigue siendo un número posible. En una dirección web el contexto ayuda solo en parte, y lo vimos en el desafío:

- Las palabras fijas se adivinaban fácil ("xttps" → "https", "featurm" → "feature").
- En el ID del video no hay contexto: "q8wgKzJw" y "q8WgKzJw" son igual de plausibles, y solo uno lleva al video correcto.
- El mensaje 3 es el caso extremo. `Pr3mi0!:)10-dIeZ` tiene forma de contraseña, y si no hubiera estado escrito en leet, ninguna de las 24 combinaciones habría sido "más legible" que otra.

Además, adivinar tiene un riesgo silencioso: un texto que "se lee bien" puede estar mal y nadie se da cuenta.

El código aporta:

- **Detección garantizada:** avisa que hubo un error aunque el texto se lea perfecto.
- **Corrección objetiva:** con un solo error, lo ubica y lo corrige sin ambigüedad.
- **Independencia del contenido:** funciona igual para texto, números, claves o datos binarios.

Reflexión 3

¿Pudieron confirmar que su corrección es correcta? ¿Cómo?

En parte, y con una salvedad importante. Después de corregir, recalculamos con `paridad.py` y todas las filas y columnas coinciden con CHECK. Pero con varios errores eso **no prueba nada**: cualquiera de las $k!$ combinaciones también deja la paridad consistente, porque cada una invierte un bit en cada fila y cada columna afectada. La confirmación vino de otros criterios:

- En los tres mensajes era la **única** combinación que daba un texto válido. Las demás dejaban caracteres imposibles en una URL (`$`, `}` o caracteres no imprimibles) o texto sin sentido.
- Los IDs de YouTube quedaron con 11 caracteres válidos y los links abren un video; un ID mal corregido lleva a un video inexistente.
- El mensaje 3 forma una frase coherente.

Una confirmación independiente del contenido sería comparar varias copias del mismo mensaje. El transmisor repetía cada mensaje, así que con varias copias con errores en lugares distintos alcanza con quedarse, bit a bit, con el valor que tiene la mayoría (votación por mayoría).

Reflexión 4

Investigar y describir con el mayor detalle posible cómo funciona la técnica de corrección Hamming (7,4) y por qué es superior a la demostración didáctica que hicimos en este Laboratorio. Luego, aplicar la técnica sobre la carga útil obtenida en el desafío y elaborar un paquete de datos que podrían transmitir para que, en caso de corrupción, pueda ser recuperado.

Cómo funciona. Hamming (7,4) toma bloques de 4 bits de datos (d_1 a d_4) y agrega 3 bits de paridad (p_1, p_2, p_3), formando palabras de 7 bits. Las paridades van en las posiciones que son potencias de 2 (1, 2 y 4). Cada una controla las posiciones cuyo número, escrito en binario, tiene ese bit en 1:

Posición	1	2	3	4	5	6	7
Binario	001	010	011	100	101	110	111
Bit	p_1	p_2	d_1	p_3	d_2	d_3	d_4

- $p_1 = d_1 \oplus d_2 \oplus d_4$ (cubre las posiciones 1, 3, 5, 7)
- $p_2 = d_1 \oplus d_3 \oplus d_4$ (cubre 2, 3, 6, 7)
- $p_3 = d_2 \oplus d_3 \oplus d_4$ (cubre 4, 5, 6, 7)

Al recibir, el receptor recalcula los tres controles sobre lo que le llegó:

$$s_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7, \qquad s_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7, \qquad s_3 = b_4 \oplus b_5 \oplus b_6 \oplus b_7$$

El número binario $s_3 s_2 s_1$ es el **síndrome**. Si vale 000 no hay error; si no, es directamente la **posición del bit dañado**, y alcanza con invertirlo. Funciona porque cada posición está cubierta por una combinación distinta de paridades, que coincide con su número en binario. La distancia mínima entre palabras válidas es 3: corrige 1 error por bloque, o detecta 2 si no se intenta corregir.

Ejemplo con el carácter "P" (ASCII 80 = `01010000`):

- Mitad alta `0101` ($d_1 = 0$, $d_2 = 1$, $d_3 = 0$, $d_4 = 1$): $p_1 = 0 \oplus 1 \oplus 1 = 0$, $p_2 = 0 \oplus 0 \oplus 1 = 1$, $p_3 = 1 \oplus 0 \oplus 1 = 0$, así que se transmite `0100101`.
- Mitad baja `0000`: se transmite `0000000`.
- Si llega `0100111` (dañada la posición 6): $s_1 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$, $s_2 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$, $s_3 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$. El síndrome $110 = 6$ indica la posición, se invierte ese bit y se recupera `0101`.

Por qué es superior a la paridad bidimensional del laboratorio.

	Paridad bidimensional	**Hamming (7,4)**
Ubicación del error	Cruce de fila y columna	El síndrome da la posición directamente
Errores corregibles	1 en todo el mensaje	1 por cada bloque de 7 bits
Con 2 o más errores	Ambiguo: hay que adivinar (nos pasó con 3 y 4)	Se corrigen todos si caen en bloques distintos
Depende del contenido	Para desempatar, sí	No
Redundancia	Baja (filas + columnas)	Alta (3 bits cada 4 de datos)

En el mensaje 3, por ejemplo, la paridad 2D agregaba 32 bits de control a 112 de datos. Hamming agrega 96 a 128, pero recupera el mensaje sin ambigüedad.

Aplicación a la carga útil del desafío. Codificamos el mensaje 3, `Pr3mi0!:)10-dIeZ` (16 caracteres), con nuestro script `hamming.py`:

1. Cada carácter ASCII se completa a 8 bits.
2. Se divide en dos mitades de 4 bits.
3. Cada mitad se codifica en una palabra de 7 bits.

Quedan 14 bits por carácter y 224 bits en total:

Car.	**ASCII (8 bits)**	**Palabra alta**	**Palabra baja**
P	01010000	0100101	0000000
r	01110010	0001111	0101010
3	00110011	1000011	1000011
m	01101101	1100110	1010101
i	01101001	1100110	0011001
0	00110000	1000011	0000000
!	00100001	0101010	1101001
:	00111010	1000011	1011010
)	00101001	0101010	0011001
1	00110001	1000011	1101001
0	00110000	1000011	0000000
-	00101101	0101010	1010101
d	01100100	1100110	1001100
I	01001001	1001100	0011001
e	01100101	1100110	0100101
Z	01011010	0100101	1011010

El paquete a transmitir, en el mismo estilo JSON que usó la cátedra (archivo `paquete_hamming.json`):

```
{"CODE": "HAMMING(7,4)", "CHARS": 16, "PL":
 "01001010000000000111101010101000011100001111001101010101
  11001100011001100001100000000101010110100110000111011010
  01010100011001100001111010011000011000000001010101010101
  11001101001100100110000110011100110010010100101101101010"}
```

- **CODE** indica el código usado.
- **CHARS** indica la cantidad de caracteres, para validar el largo ($16 \times 14 = 224$ bits).
- **PL** son los bits codificados. Acá están partidos en cuatro líneas de 56 bits solo para que entren en la página; en el paquete real van en una sola cadena.

Prueba de recuperación. Invertimos un bit en cuatro bloques distintos (los bits 22, 70, 145 y 176 del paquete) y decodificamos con `python hamming.py`. El script corrigió los cuatro errores (bloque 4 posición 1, bloque 10 posición 7, bloque 21 posición 5 y bloque 26 posición 1) y recuperó `Pr3mi0!:)10-dIeZ` sin ambigüedad. Con la paridad bidimensional, esos mismos cuatro errores habrían dejado 24 combinaciones posibles, igual que en el desafío.